Online Q&A Session 3

Git it done...

These notes contain information from Q&A Sessions on the following dates:

- Friday 16/02/2024
- Friday 23/02/2024

Contents

- Git it done...
  - Requests for Help?
  - Overview
- Before you start (Required Software)
  - Microsoft Terminal Installation
  - Git Installation
  - Laragon Installation
- The CLI
  - Parts of the Prompt
  - Setting up Microsoft Terminal to use Git Bash as a CLI
- Repositories and Folders
  - Creating a Source/Repos folder structure for projects
- Git Started
  - Initialising a Repository for a 'project'
  - Verify Repository was Created
  - Creating a ReadMe.md for the project
  - Checking the Status of the Repo
  - Adding a file for tracking
  - Committing a file to history
  - History so Far...
  - Global configuration changes
- Shaking the Tree

- Creating a branch
- Switching Branches
- Merging a Branch into "main"
- EEK! I've got Conflicts!

Requests for Help?

If at all possible, please send any requests for explanation to the Help Desk before 08:30 on Friday so as to allow Adrian to prepare a demonstration or similar.

Overview

In this set of notes...

- Creating a Source\Repos folder structure for projects
- Setting up Microsoft Terminal to use Git Bash as a CLI
- Initialising a Repository for a 'project'
- Creating a ReadMe.md for the project
- Adding a file for tracking
- Committing a file to tracking
- Global Configuration of Git User Name and Email Address
- and more...

Before you start (Required Software)

You will need to ensure one of the following combinations is installed:

- Microsoft Terminal and Git-SCM, or
- Microsoft Terminal and Laragon

Microsoft Terminal Installation

This is a great way to interact with the command line.

Open the Microsoft Store, and search for "Microsoft Terminal".

Install the "Microsoft Terminal" version and not the preview.

Git Installation

Easiest way is to download and use the Git SCM installer:

https://git-scm.com/download/win

Use the 64 Bit version when possible.

Run the installer and accept the defaults.

Laragon Installation

Download the Full Installer from https://laragon.org/download/index.html.

Run the installer and accept the defaults.

This places Laragon in the folder `c:\Laragon`.

The CLI

CLI stands for Command Line Interpreter.

There are many of them including:

- Windows Command Prompt
- Microsoft PowerShell
- bash
- zshell
- and many more

Parts of the Prompt

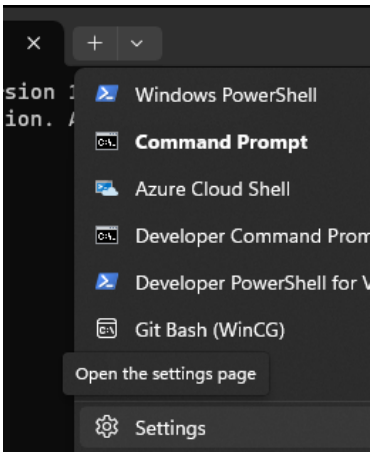
Image	Description
AD+5001775@NMT-204426	This is the user account, followed by the computer name and separated by an `@` Literally: *User **at** Computer*
MINGW64	Tells us that we are running in MinGW 64 bit
~/Source/Repos	Shows we are in the `~/Source/Repos` folder, where `~` is the user's (your) home folder. In Windows terms, this is the same as: `C:\Users\UserName\Source\Repos`

Setting up Microsoft Terminal to use Git Bash as a CLI

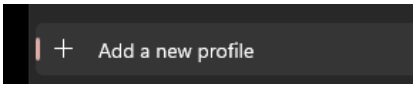
Open MS Terminal.

Click on the `v` (or press `CTRL` + `,`) on the tab bar.

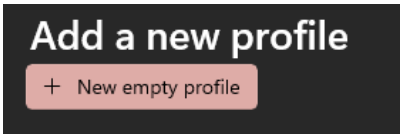
Click *Settings*...



Click on *Add a new profile*...

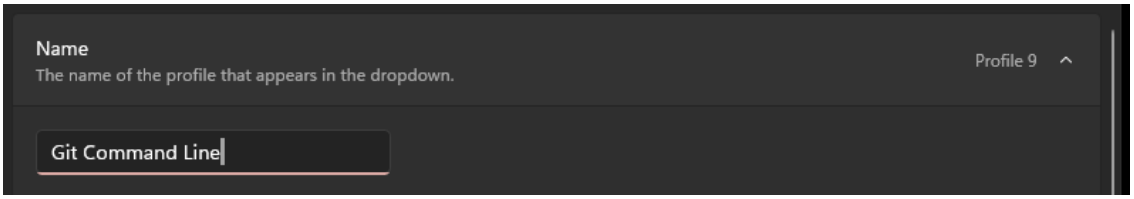


Click *New Empty Profile*...



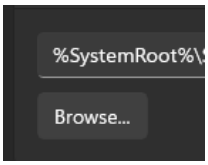
Click on the `v` next to the "*Profile x*"...

And give the profile a name...



Next find the *Command Line* entry and click on the `v`

Then click the *Browse* button...

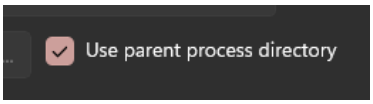


At home, if you have installed using Git-SCM installer then the default location for Git is: `c:\program files\git`. If you have Laragon installed using the default installation location, then you should find Git in the `C:\Laragon\bin\git` folder.

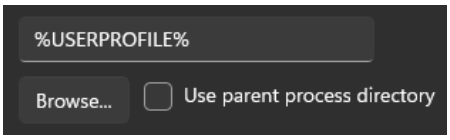
> At TAFE, we have a different location, `c:\ProgramData\git` or even `C:\ProgramData\Laragon\bin\git`.

Use the browse to locate the folder and find either the `git.exe` or the `bash.exe` file in the `bin` folder.

Click on the Starting Directory `v` and then untick the "*Use parent process directory*"...



This should then show `%USERPROFILE%` in a text field and `Browse` button underneath...



No matter what is shown, clear the content and enter the following in its place...

`%USERPROFILE%\Source\Repos`

Click `SAVE` to save the changes

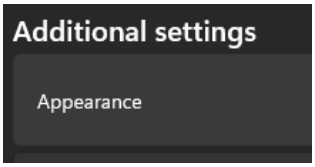
Visual flair (optional)

You may optionally customise the appearance of the terminal.

Adrian tends to do this to visually identify the various prompts.

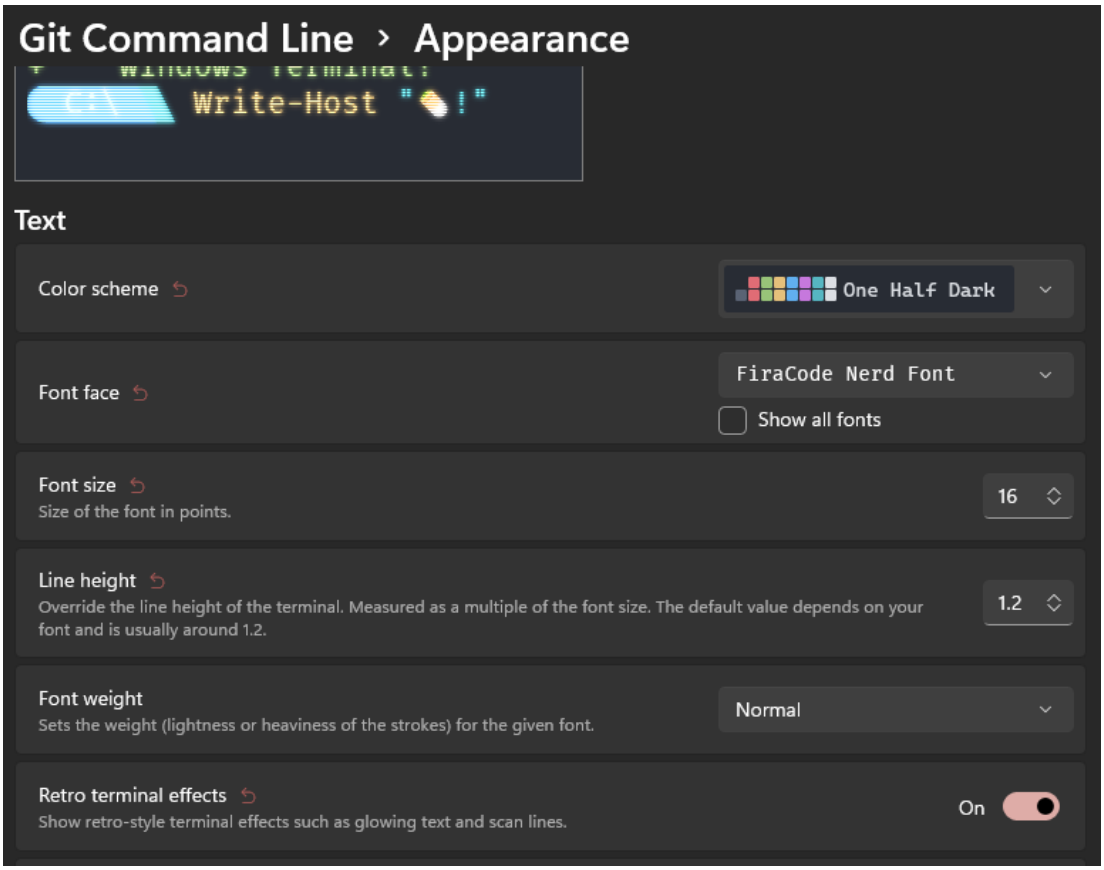
The steps below are optional, and you may vary to your own "taste".

Click the *Appearance* item.



You may change the base colour scheme, the font face and more.

Here are the settings Adrian has for this terminal prompt:



Click Save then you can click the `X` next to settings to close the dialog.

Finally you are able to open the new terminal by clicking on the `v` and selecting your custom terminal or using the short-cut it shows on the right next to your custom terminal.

You will now have your terminal open and ready:

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos
$
```

Repositories and Folders

Projects are contained in a folder in your user's root/home folder.

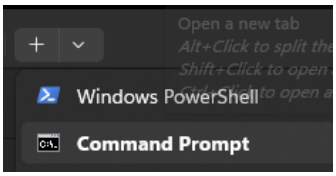
This folder is the `Source/Repos` folder.

If you do not have this folder then the steps below should be followed.

Creating a `Source/Repos` folder structure for projects

Open Terminal and then click the `v` next to the `+` on the tab bar.

Select the *Command Prompt* option



When the terminal opens use the following commands to change into your "user home" and make the `Source\Repos` folder.

> **Important:** We tend to use the forward slash `/` in notes as they show up easily, plus most operating systems other than Windows use the `/` as the folder separator.
>
> Later you will see this in action when using the Git command line.

Here are the commands to use:

```
cd %userprofile%
mkdir Source
mkdir Source\Repos
```

You should now have the folders created.

Check this using:

```
dir Source\Repos
```

It should show a folder listing with no files.

Git Started

We are now ready to get started, and create a repository and work with it.

Initialising a Repository for a 'project'

There are three main ways to start a project. Two of them presume you have no existing files to begin with, and the third when you have files in an existing folder.

Option 1: Create a folder then initialising

Use `mkdir` command to create the folder, change into the new folder, and then using the `git init .` command.

Let's presume the new project will be in the `Duck-Rogers` folder.

Use the `pwd` command to verify the folder:

```
$ pwd
/c/Users/5001775/Source/Repos
```

You see that the Windows `\` have been reversed to `/` and the `:` removed.

```
mkdir Duck-Rogers
cd Duck-Rogers
git init .
```

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos
$ mkdir Duck-Rogers

AD+5001775@NMT-204426 MINGW64 ~/Source/Repos
$ cd Duck-Rogers/

AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers
$ git init .
Initialized empty Git repository in C:/Users/5001775/Source/Repos/Duck-Rogers/
.git/
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$
```

The command prompt will now show either `(master)` or `(main)` - if it is `master`, we will show you later how to (a) change it to `main` for the current

project and (b) to make `main` the default.

Option 2: Initialise project and folder at same time

To create the `Duck-Rogers` folder and initialise in a single move, we do the following:

```
git init Duck-Rogers
cd Duck-Rogers
```

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos
$ git init Duck-Rogers
Initialized empty Git repository in C:/Users/5001775/Source/Repos/Duck-Rogers/
.git/
```

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos
$ cd Duck-Rogers/

AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ ls
```

Option 3: Initialise project with existing files

The best part of this is that you are basically able to use the previous options, except the folder already exists.

For example, if `Duck-Rogers` already exist and has files, then, if we are at the parent folder for Duck-Rogers we can use the `git init Duck-Rogers` command.

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers
$ ls -la
total 8
drwxr-xr-x 1 AD+5001775 1610613249 0 Feb 16 11:40 .
drwxr-xr-x 1 AD+5001775 1610613249 0 Feb 16 11:36 ..
-rw-r--r-- 1 AD+5001775 1610613249 0 Feb 16 11:40 Hello.txt

AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers
$ cd ..

AD+5001775@NMT-204426 MINGW64 ~/Source/Repos
$ git init Duck-Rogers
Initialized empty Git repository in C:/Users/5001775/Source/Repos/Duck-Rogers/
.git/

AD+5001775@NMT-204426 MINGW64 ~/Source/Repos
$ ls -la Duck-Rogers
total 12
drwxr-xr-x 1 AD+5001775 1610613249 0 Feb 16 11:41 .
drwxr-xr-x 1 AD+5001775 1610613249 0 Feb 16 11:36 ..
drwxr-xr-x 1 AD+5001775 1610613249 0 Feb 16 11:41 .git
-rw-r--r-- 1 AD+5001775 1610613249 0 Feb 16 11:40 Hello.txt
```

If you are in the `Duck-Rogers` folder and there are files in the folder then using the `git init .` command will do the same thing.

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers
$ ls -la
total 8
drwxr-xr-x 1 AD+5001775 1610613249 0 Feb 16 11:44 .
drwxr-xr-x 1 AD+5001775 1610613249 0 Feb 16 11:36 ..
-rw-r--r-- 1 AD+5001775 1610613249 0 Feb 16 11:40 Hello.txt

AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers
$ git init .
Initialized empty Git repository in C:/Users/5001775/Source/Repos/Duck-Rogers/
.git/
```

Verify Repository was Created

To check it is created we use the `ls -la` command:

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ ls -la
total 12
drwxr-xr-x 1 AD+5001775 1610613249 0 Feb 16 11:36 .
drwxr-xr-x 1 AD+5001775 1610613249 0 Feb 16 11:36 ..
drwxr-xr-x 1 AD+5001775 1610613249 0 Feb 16 11:36 .git
```

Notice the last line. The `d` at the start tells you it is a folder (directory), and its name is `.git`. It is a hidden folder.

All tracking of changes are then stored in the `.git` folder. If you delete this folder, you delete the whole history of your project.

Creating a `ReadMe.md` for the project

The Bash/git shell has the ability to create a empty file really easily.

For example an empty `ReadMe.md` file would be created using:

```
touch ReadMe.md
```

In the Windows Command Prompt we need to use:

```
copy nul > ReadMe.md
```

The `>` tells the command to write to a file with the name of the file after the `>`.

If you used `>>` it would append the output of the command to the file named after the `>>`.

Checking the Status of the Repo

Checking the status of the repository is done using the `git status` command:

```
git status
```

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Hello.txt
        ReadMe.md

nothing added to commit but untracked files present (use "git add" to track)
```

In this example, we have two files that are not "tracked" by the repository.

Untracked files are shown in Red, tracked in green.

Adding a file for tracking

To add a file to be tracked we use `git add FILENAME`. For example:

```
git add ReadMe.md
```

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git add ReadMe.md
```

Checking the status after adding shows:

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   ReadMe.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Hello.txt
```

Once a file is tracked, any time you make changes and are happy with them, you should perform a `git status` and then `git add` the changed file.

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   ReadMe.md

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   ReadMe.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Hello.txt
```

Here you see the file has been modified.

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git add ReadMe.md

AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   ReadMe.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Hello.txt
```

After adding the changes, we see it is back to 'new'.

Committing a file to history

Committing files to the repo, makes them available as a chapter in the project's history.

To commit changes we use:

```
git commit -m "Commit message"
```

In the above line the `-m` indicates that we are including a message.

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git commit -m "init: Start of Project Duck-Rogers, includign ReadMe.md"
[main (root-commit) 3dfbed6] init: Start of Project Duck-Rogers, includign Rea
dMe.md
 1 file changed, 3 insertions(+)
 create mode 100644 ReadMe.md
```

Now our `git status` should show:

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Hello.txt

nothing added to commit but untracked files present (use "git add" to track)
```

The committed file is still tracked, but as no changes have been made it does not show.

> **Best Practice Hint:** Note in my message, that I included `init:` at the start. This is telling me (and anyone who is on the project) that it is the initialisation of the project.
>
> If it is a new feature we use `feat` instead and often would add the feature name in `( )` round brackets. More on this later.

Exercise

Run through the steps above.

Once complete, edit the `ReadMe.md` and add:

```
## Summary
- Author: Adrian Gould
- Version: 1.0
```

Save the changes, then check the status of the repository.

It should show the `ReadMe.md` has changed:

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   ReadMe.md
```

Add the changes to the repo and **commit** them using the message:

```
feat(readme): Update ReadMe.md to include author and version
```

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git commit -m "feat(readme): Update readme to include author and version"
[main cf0fa26] feat(readme): Update readme to include author and version
 1 file changed, 7 insertions(+)
```

History so Far...

To see the history of the project so far we use...

```
git log
```

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git log
commit cf0fa26b1ea9ecd4c20ec08e3d0c7648ff57fa1d (HEAD → main)
Author: AdyGCode <Adrian.Gould@nmtafe.wa.edu.au>
Date:   Fri Feb 16 12:37:47 2024 +0800

    feat(readme): Update readme to include author and version

commit 3dfbed65d754c88bc4ba77096bb4c3c67ececf4a
Author: AdyGCode <Adrian.Gould@nmtafe.wa.edu.au>
Date:   Fri Feb 16 12:28:31 2024 +0800

    init: Start of Project Duck-Rogers, includign ReadMe.md

AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$
```

> **Note:** The history is shown in reverse order... newest first.

This is very lengthy, and most of the time we can work with a short version of the log. To get this we use:

```
git log --oneline
```

Which produces a shorthand version of the log:

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git log --oneline
cf0fa26 (HEAD → main) feat(readme): Update readme to include author and version
3dfbed6 init: Start of Project Duck-Rogers, includign ReadMe.md
```

What is important is that the message shows in this log. It is a good idea *not* to have a very long first line to a commit message.

Global configuration changes

Git, like any software has many things that can be changed. These area stored in a global configuration file, in a project only configuration file.

Most of the configuration options are able to be set for both, but we show the global (for you as the user) changes you may want to make.

Git User Name and Email Address

To change the username and email address being stored as part of the commit sequence, we use the following two commands:

```
git config --global user.name "YOUR NAME"
git config --global user.email "YOUR EMAIL ADDRESS"
```

The `--global` indicates that the changes are stored for this and all future projects.

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git config --global user.name "Adrian Gould"

AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git config --global user.email "adrian.gould@nmtafe.wa.edu.au"
```

To see the global settings...

```
git config --list --global
```

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git config --global --list
credential.helper=cache --timeout=86400
user.name=Adrian Gould
user.email=adrian.gould@nmtafe.wa.edu.au
init.defaultbranch=main
```

Per Project Configuration

These need to be repeated for any project that is not going to be listed under your 'global' config.

Make sure you are in the project you wish to use project only configured settings.

Then set your required configuration by using:

```
git config SETTING VALUE
```

For example, changing the "user" and "email" to be your TAFE details rather than using your personal email and name...

Lets say your ID is `00000000` and email is `00000000@tafe.wa.edu.au` we could use:

```
git config user.name "00000000"
git config user.email "00000000@tafe.wa.edu.au"
```

The `--global` indicates that the changes are stored for this and all future projects.

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git config --global user.name "Adrian Gould"

AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git config --global user.email "adrian.gould@nmtafe.wa.edu.au"
```

Default "Branch" name to Main...

To do this for ALL FUTURE projects we use:

```
git config --global init.defaultBranch main
```

Change Current Project's Branch Name

To change the current 'master branch' to be the 'main branch' we need to do the following *AFTER* a commit is completed:

```
git branch -m master main
```

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (master)
$ git branch -m master main

AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
```

Did you forget `-m` on `git commit`?

If you type in `git commit` and do not add the `-m` with the message then the `vi` editor is shown.

The easiest way out is to do the following:

- **Option 1:** Press `ESCAPE` then type `:qa` - This is a quit and abandon changes. This is the better option.
- **Option 2:** Press `ESCAPE`, then press and hold the `SHIFT` whilst typing `Z` twice - This is a save and quit

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git commit
Aborting commit due to empty commit message.
```

Default Commit Message Editor

It is possible to change the code editor for commit messages to something like Notepad.

Below we change the editor to "nano" - a extremely small footprint terminal text editor on MacOS and Linux.

```
git config --global core.editor "nano"
```

To change it to Notepad we would use:

```
git config --global core.editor "notepad"
```

> **Important:** when using a GUI editor such as Notepad, you **MUST SAVE** and **CLOSE** the message, then (usually optionally) **QUIT** the application for the message to be transferred to the git command.

Now when you enter `git commit` the Notepad editor will open. The command line will tell you it is waiting for the editor to close a file (`COMMIT_MESSAGE`).

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git commit
hint: Waiting for your editor to close the file... unix2dos: converting file C:/Use
rs/5001775/Source/Repos/Duck-Rogers/.git/COMMIT_EDITMSG to DOS format...
```

When Notepad is opened, it will show a set of comments that tell you details about the added files as comments.

```
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch main
# Changes to be committed:
#       modified:   ReadMe.md
#
```

Now editing the commit message and adding the details that may be more than just a one liner is easy.

You then enter the SHORT commit message and, as required, a blank line followed by more details about the changes that have been made to the project.

For example:

```
feat(readme): Add demo commit demo of external editor

- Change config for default commit editor to Notepad
- Add a single blank line to the ReadMe for demo purposes
- Add ReadMe to staging
- Run git commit
- Demonstrate the long form of commit message
- Save (`CTRL`+`S`) and close (`CTRL`+`F4`) the message
- Optionally, quit the editor (`ALT`+`F4`)
```

> **Warning:** Tests have indicated that you will not be able to use 'VSCode' as the editor for messages.
>
> On execution the editor does not negotiate correctly with the command line and git aborts the commit.

Shaking the Tree

Git allows us to try things out without affecting our working code. This ability is known as branching.

The main branch is sometimes called the 'trunk', and like any tree trunk, it has branches that split off from it.

Unlike trees, branches in version control may merge back into the trunk.

But this is not just for the trunk, a branch may have branches split off from it, and allow for testing on an idea before it is added back into the branch if the idea solves that particular problem.

Creating a branch

We usually create a branch when we want to work on an idea that is not certain to work, or if we need to test an idea.

When possible this is done after a commit, but it may be done at any time, if needed.

To create a branch we use:

```
git branch BRANCH_NAME
```

For example:

```
git branch feat-readme
```

Switching Branches

Next we need to switch to the branch to work on it:

```
git switch BRANCH_NAME
```

For example:

```
git switch feat-readme
```

You are now able to work on the code in this new branch.

At any point you have code that is working, you may add to the staging area and commit the file or files.

Merging a Branch into "main"

To merge the branch you were working on into the main branch, we do the following steps:

```
git switch main

git merge BRANCH_NAME
```

For example:

```
git switch main
git merge feat-readme
```

EEK! I've got Conflicts!

Not all merges will go smoothly, and you may end up with conflicts in your code.

This often happens when you have been working on the main branch and your development branch and have committed changes on both.

To resolve this we need to edit the file(s) with the errors and fix the changes by accepting the code that is correct, or most relevant to the project.

The image below shows the conflict being reported.

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git merge demo-branch
Auto-merging ReadMe.md
CONFLICT (content): Merge conflict in ReadMe.md
Automatic merge failed; fix conflicts and then commit the result.
```

To resolve the conflicts you now edit the file (or files) with the conflicts.

Opening the file will reveal the conflicting code highlighted using `<<<<<<` and `>>>>>>` to show the 'direction' the changes come from (and the branch), and `======` to show where the changes end/start.

Here is an example:

```
<<<<<<< HEAD

Text from the version in the HEAD (main) branch

=======

Text from the version in the feature-1 branch

>>>>>>> feature-1
```

Opening the required file and accepting lines of code (leaving them in) and rejecting lines of code (deleting them) helps resolve the conflict.

The final steps of this resolution are to remove the `>>>>>>`, `======` and the `>>>>>>` lines, save the changes and close the file.

At this point you are able to add and commit the now resolved conflict:

```
git add FILE_NAME
git commit -m "MERGE SUMMARY MESSAGE"
```

The image below shows a really bad commit message:

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main|MERGING)
$ git add ReadMe.md

AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main|MERGING)
$ git commit -m "Test"
[main 6518741] Test
```

If we now log the commits we will see a diagram of the branches with the conflict shown in red:

```
AD+5001775@NMT-204426 MINGW64 ~/Source/Repos/Duck-Rogers (main)
$ git log --oneline --decorate --graph --all
*   6518741 (HEAD → main) Test
|\
| * 631b7ba (demo-branch) feat(readme): merge conflict?
* | a0965e7 feat(readme): test merging
|/
* f888e2d feat(readme): Add demo branch text
* 7512236 feat(readme): Add demo commit demo of external editor
* 32bd69e Test: Test commit only
* cf0fa26 feat(readme): Update readme to include author and version
* 3dfbed6 init: Start of Project Duck-Rogers, includign ReadMe.md
```